

## ✓ Project Name:

### Artisan Marketplace Dashboard

---

## ✓ Project Goal (What is the project about?):

You are building an **AI-powered platform for Artisans** where artisans can register, showcase their handmade products, manage orders, and interact with customers. The main goal is to help artisans have an online presence, manage their products, and use AI tools to improve their business.

---

## ✓ Key Features of the Project:

### 1. User Authentication:

- Artisans can **register** and **login** securely.
- Supabase Authentication is used for managing login, registration, and session management.

### 2. Artisan Dashboard:

- After login, artisans get their **personal dashboard**.
- They can:
  - Add products
  - View orders
  - Manage product stock
  - See analytics about product views or sales

### 3. Product Management:

- Artisans can upload products with details like:
  - Name
  - Description
  - Price
  - Image
  - Eco Score
- Products are stored in Supabase Database.

#### **4. Order and Review Management:**

- Customers can place orders for artisan products.
- Artisans can view and manage orders.
- They can also manage reviews and ratings.

#### **5. AI Integration (For Next Steps):**

- AI tools (from Google Cloud AI or others) will be integrated later.
- Example use cases:
  - Automatically suggest product categories
  - Automatically detect counterfeit products using image analysis
  - Provide analytics and suggestions based on sales data

---

## **What You Have Done So Far (Step-by-Step Overview):**

### **Step 1: Database Integration**

- Used **Supabase as backend service**.
  - Created tables like `Artisan`, `Product`, `Order`, `Review`, and `SupportQuery`.
  - Tables are connected via foreign keys where needed.
- 

## ✓ Step 2: Supabase Client Setup

- Supabase client initialized in `supabaseClient.js`.
  - Supabase URL and API key configured securely.
- 

## ✓ Step 3: Replacing Mock Data with Supabase API Calls

- Replaced mock data fetching and authentication logic with **real Supabase API calls**.
  - Used Supabase methods to:
    - Fetch products from DB
    - Create new product entries
    - Authenticate users (register and login)
    - Manage user sessions
- 

## ✓ Step 4: AuthContext Integration

- Implemented global authentication state with React Context.
  - Provided methods: `login`, `register`, `logout`, and session management via Supabase.
-

## ✅ Step 5: UI Implementation

- Built pages such as:
    - Login
    - Register
    - Home
    - Dashboard
    - Add Product
    - Profile Settings
  - Dynamic display of products and user data from Supabase database.
- 

## ✅ Step 6: Testing Data Persistence

- Confirmed data persists in Supabase DB after page refresh or new logins.
  - Verified frontend displays correct data fetched from Supabase.
- 

## ✅ Next Steps (Planned but not done yet):

1. ✅ Fix all minor issues (authentication bugs, 404 errors, loading issues).
2. ✅ Ensure all Supabase keys and URLs are set correctly.
3. ✅ Integrate Google Cloud AI tools (required for evaluation):
  - Example: AI for counterfeit detection, smart analytics, category suggestions.
4. ✅ Prepare a Prototype Deck (PPT) and Video for Submission.
5. ✅ Test thoroughly in multiple environments to ensure everything works smoothly.

---

## ✅ Why This Plan Makes Sense:

- You are building a **real full-stack application** with a scalable backend (Supabase).
- The frontend is made with **React + TypeScript + Vite**.
- It supports dynamic data fetching and manipulation.
- Designed for easy future integration of AI tools.
- Proper separation of concerns:
  - Authentication logic in context.
  - Supabase client as a central API layer.
  - Clean UI flow for artisans.

---

## ✅ Important Notes:

- Supabase handles most backend work → No need to manage a custom server.
- No AI features are fully integrated yet, but the system is ready for that.
- All data is persisted in Supabase database correctly and dynamically.

---

## ✅ Final Summary:

Your project is a **functional, modular, scalable Artisan Marketplace Dashboard** with full authentication, product management, order management, and profile management powered by Supabase. The next big milestone is adding AI features and submitting a clear prototype (deck + video + working app link).

---

Let me know if you want me to create a summary document (like a PPT-ready version) for your submission deck.

**You said:**

Now tell me what we have done so far and what We want to be done

**ChatGPT said:**

Here's a clear, structured summary of **what we have done so far** and **what still remains to be done** in your project:

---

## ✓ ✓ What We Have Done So Far:

### 1 Project Setup

- Set up frontend using **React, TypeScript, Vite**.
  - Supabase used as backend service for authentication and database.
- 

### 2 Database Setup (Supabase)

- Created the following tables:
    - Artisan
    - Product
    - Order
    - Review
    - SupportQuery
  - Ensured correct relationships and constraints.
- 

### 3 Supabase Client Configuration

- Configured Supabase URL and Anon Key in `supabaseClient.js`.
  - Fixed syntax errors in Supabase setup.
- 

#### 4 Authentication Integration

- Implemented **AuthContext** for global auth state.
  - Supabase API methods for:
    - Register
    - Login
    - Logout
    - Session management
  - Ensured token-based session persists.
- 

#### 5 Pages Developed

- Login Page
- Register Page
- Home Page
- Dashboard Page
- Add Product Page
- Profile Settings Page

All integrated with real Supabase calls instead of mock data.

---

## 6 Components Updated

- Header
- Sidebar
- ProtectedRoute
- ProductModel
- ReviewRating
- Analytics
- DashboardOverview
- ProfileSettings

All components dynamically fetch data from Supabase and display correctly.

---

## 7 Fix Supabase Keys

- Replaced Supabase URL and Anon Key in all files where needed.
- 

## 8 Project Runs Locally

Successfully run the project using:

```
npm install  
npm run dev
```

- - Localhost opens, dashboard loads (with some limitations).
- 



## What Remains to Be Done:



## 1 Fix Remaining Issues

- Resolve 401 errors (authentication token not correctly handled).
  - Resolve UI bugs where dashboard fails to load data or logs out unexpectedly.
  - Resolve other warnings (missing language modules, unused preloads).
- 

## 2 AI Integration (Required by Submission Guidelines)

- Integrate **Google Cloud AI tools** such as:
  - Product counterfeit detection (image analysis).
  - Smart product category suggestions.
  - Sales analytics or predictions.

This is essential for the technical merit evaluation (40% weightage).

---

## 3 Prepare Submission Artifacts

-  Prototype Deck (PPT) → Well-structured, describing project purpose, architecture, features, AI integrations, and future plans.
  -  Demo Video (3 minutes) → Showing user flow, registration, login, dashboard, product management, and AI features if possible.
  -  Working Prototype Link → Deploy and host the app so evaluators can test live.
  -  GitHub Repository → Should contain clean, well-documented code with proper commits.
- 

## 4 Final Testing

- Test:
    - Registration & login flows.
    - Product CRUD operations.
    - Order placement and management.
    - Profile settings.
    - Proper data is stored in Supabase DB.
    - AI features integrated and working as expected.
- 

## Summary

| Phase                            | Status                                                                                        |
|----------------------------------|-----------------------------------------------------------------------------------------------|
| Project setup                    |  Done      |
| Supabase DB design               |  Done      |
| Supabase client setup            |  Done      |
| Auth flow implementation         |  Done      |
| Pages and components implemented |  Done      |
| Supabase key config              |  Done      |
| Run project locally              |  Done      |
| Fix bugs & warnings              |  Remaining |
| Integrate AI tools               |  Remaining |
| Create Prototype PPT + Video     |  Remaining |

---

👉 Now we are at the point where the **basic prototype is working** with some limitations.

👉 Next big focus should be:

1. Fix auth/session bugs → ensure reliable login flow
2. Fix warnings/errors in console → improve code quality
3. Integrate AI tools → to meet evaluation criteria
4. Prepare final prototype deck and video

---

Let me know if you want me to help step by step from here.

Shall we start by fixing the authentication 401 issue first or proceed to AI integration planning?

**You said:**

Okay now give the list of features what we have done For that I want the workflow for all the features For each features I want workflow and the technology we have used

**ChatGPT said:**

Here is a structured and detailed list of **features implemented so far**, along with their **workflows and technologies used**:

---

## ✅ 1 User Registration & Login Feature

☀️ **Workflow:**

1. User visits the **Register Page** → enters name, email, password, and phone.
2. Data is sent via Supabase Auth API → `supabase.auth.signUp()`.
3. Upon successful registration → redirected to **Login Page**.

4. User enters email & password on **Login Page** → Data sent via Supabase → `supabase.auth.signIn()`.
  5. On success → Auth context stores session → Redirects to **Dashboard Page**.
- 

### Technology Used:

- **React Context API** → Manage authentication state.
  - **Supabase Auth API** → signUp, signIn, signOut, session management.
  - **React Router DOM** → Navigation between pages.
  - TypeScript → Type safety.
- 

## **Dashboard Display Feature**

### Workflow:

1. After successful login → User lands on **Dashboard Page**.
  2. Components fetch dynamic data from Supabase tables:
    - Products
    - Orders
    - Reviews
  3. Supabase `from('table').select('*')` → Data is displayed in graphs and tables.
  4. Sidebar navigation → Switch between different dashboard sections.
- 

### Technology Used:

- Supabase Client API → Data fetching.
  - Recharts → For graphs & analytics.
  - React Components → DashboardOverview, Analytics, ReviewRating.
  - TypeScript + Vite.
- 

## **Add Product Feature**

### **Workflow:**

1. User clicks **Add Product** button → Opens Add Product Modal.
  2. Fills product info → clicks submit.
  3. Data sent to Supabase → `from('product').insert({...})`.
  4. Product is saved in DB → State updated and product appears in the dashboard.
- 

### **Technology Used:**

- Supabase API → Insert operation.
  - React Modal Component.
  - State Management → React useState + useContext.
  - TypeScript.
- 

## **Profile Settings Feature**

### **Workflow:**

1. User navigates to **Profile Settings Page**.
  2. Displays user data (name, email, phone).
  3. User updates info → Clicks Save.
  4. Data sent to Supabase → `from('artisan').update({...}).eq('id', userId)`.
  5. Updated info saved in DB → UI reflects changes.
- 

### **Technology Used:**

- Supabase Client API → Update operation.
  - React Forms + Context API.
  - TypeScript.
- 

## **Review and Rating Feature**

### **Workflow:**

1. User navigates to **Review & Rating Section**.
  2. Displays existing reviews fetched from Supabase → `from('review').select('*')`.
  3. User can add new review.
  4. Data sent to Supabase → `from('review').insert({...})`.
  5. Review is saved → Appears in the list immediately.
-

### ⚙️ Technology Used:

- Supabase API → Fetch & Insert data.
  - React Components → ReviewRating Component.
  - TypeScript.
- 

## ✅ 6 Order Management Feature (Optional/Planned)

### 🔆 Planned Workflow:

1. User browses products → Places an order.
  2. Order is saved in Supabase → `from('order').insert({...})`.
  3. Orders displayed in Dashboard → Admin can manage orders.
- 

### ⚙️ Technology Used:

- Supabase API → Insert + Select.
  - React Components.
  - TypeScript.
- 

## ✅ 7 Supabase Authentication & Session Management

### 🔆 Workflow:

1. On every page load → `supabase.auth.session()` checks if session exists.

2. If session found → AuthContext sets user data.
3. Provides persistent login state until logout.

---

### ⚙️ Technology Used:

- Supabase Auth API → `session()`, `signIn()`, `signOut()`, `signUp()`.
- React Context API.
- TypeScript.

---

### ✅ Summary of Technologies in Use

| Feature             | Technology                               |
|---------------------|------------------------------------------|
| Authentication      | Supabase Auth, React Context, TypeScript |
| Product Management  | Supabase Tables, React Components        |
| Profile Settings    | Supabase Update, React Forms             |
| Review & Rating     | Supabase Tables, React Components        |
| Dashboard Analytics | Supabase Data Fetch + Recharts           |
| Routing             | react-router-dom                         |
| Project Build       | Vite + TypeScript                        |
| State Management    | React Context API                        |

---

👉 Now that we have this structured list of implemented features and their workflows, we are well-prepared for next steps:



1. Fix current issues (authentication token, dashboard loading).
2. Integrate AI tools (mandatory for hackathon evaluation).
3. Final testing → Ensure data flows dynamically to/from Supabase without hardcoded mock data.

Let me know if you want me to start helping with the next step: fixing the authentication 401 issue and session persistence first.